

LESSON 1 Computer and Programming Concept

OVERVIEW

This lesson introduces you to computers, software or programs, problem solving and how to write programs/instructions using algorithm: flowcharting and pseudo coding specifically.

Objectives

After completion of this Instructional Material you should be able to:

- Understand theoretical concepts in programming
- Utilize the algorithm tools in writing programs/instructions
- Learn the different program control structures
- Become a better programmer in your own programming language

COURSE MATERIAL:

Watch:

[Programming Language Concepts](https://www.youtube.com/watch?v=nV1HAbGvQMA)
<https://www.youtube.com/watch?v=nV1HAbGvQMA>

[Algorithm using Flowchart and Pseudo code Level 1 Flowchart](https://www.youtube.com/watch?v=vOEN65nm4YU)
<https://www.youtube.com/watch?v=vOEN65nm4YU>
[Flow Chart and Pseudocode](https://www.youtube.com/watch?v=O8vPR3zh5go)
<https://www.youtube.com/watch?v=O8vPR3zh5go>

Read:

[Concept of Hardware and Software](https://www.tutorialspoint.com/computer_concepts/computer_concepts_hardware_software_concept.htm)
https://www.tutorialspoint.com/computer_concepts/computer_concepts_hardware_software_concept.htm

Computer

A **computer** is an electronic device that accepts data from the user, processes it, produces results, displays them to the users, and stores the results for future usage.

Data is a collection of unorganized facts & figures and does not provide any further information regarding patterns, context, etc. Hence data means "unstructured facts and figures".

Information is a structured data i.e. organized meaningful and processed data. To process the data and convert into information, a computer is used.

Hardware and Software

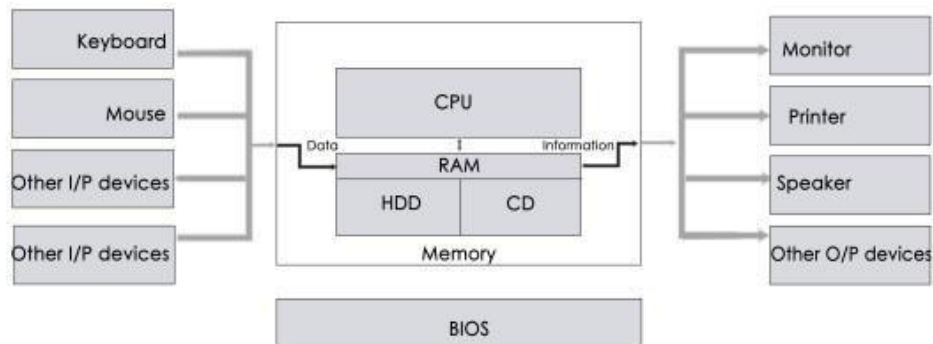
Hardware

The term hardware refers to mechanical device that makes up computer. Computer hardware consists of interconnected electronic devices that we can use to control computer's operation, input, and output. Examples of hardware are CPU, keyboard, mouse, hard disk, etc.



Hardware Components

Computer hardware is a collection of several components working together. Some parts are essential, and others are added advantages. Computer hardware is made up of CPU and peripherals as shown in image below.



Software

A set of instructions that drives computer to do stipulated tasks is called a program. Software instructions are programmed in a computer language, translated into machine language, and executed by computer. Software can be categorized into two types –

- System software
- Application software

System Software

System software operates directly on hardware devices of computer. It provides a platform to run an application. It provides and supports user functionality. Examples of system software include operating systems such as Windows, Linux, Unix, etc.

Application Software

An application software is designed for benefit of users to perform one or more tasks. Examples of application software include Microsoft Word, Excel, PowerPoint, Oracle, etc.

Differences between Software and Hardware are sorted out below –



Program and Programming

A computer is a tool used for processing data into information. The processing is controlled by a set of detailed step by step instructions. The list of instructions that the computer must follow to

be able to process data into information is called program or software. These instructions consist of statements written in a specific programming language. When a program is written, we are basically solving a problem. To solve a problem, we need to use our mind power or logic to develop the detailed instructions.

The procedure of developing the detailed instructions for the computer is called programming. It is a five-step process namely:

1. Define the problem
2. Design the solution/program
3. Code the program
4. Test the program
5. Document the program

The problem definition step means that there should be clear understanding of the problem. It requires performing the following:

1. Specify program objectives and identify the users of the program.
2. Specify the output requirements.
3. Specify the input requirements.
4. Specify processing requirements.
5. Study feasibility of implementing the program.
6. Document the analysis.

In designing the program, the following steps are done:

1. Determine the program logic through top-down approach and modularization, using hierarchy charts.
2. Design details using pseudo-code and /or flowcharts.
3. Test design with a structural walkthrough. Structural walkthrough consists of reviewing process with other programmers and system analyst and scrutinize ("walkthrough") the programmer's work.

Program coding is the step which translates the logic of the program based on the pseudo-code (algorithm) or flowcharts into a high-level programming language using the syntax and semantics of the specific language.

The steps include:

1. Determine the appropriate programming language to use.
2. Code the program in the specific programming language decided on.

Program testing consists of desk checking, debugging, and running real data to make sure that the program works. Desk checking is just reading through or checking the program manually to ensure that it is free from errors and the logic works. Debugging means detecting, locating, and removing errors in the computer program. Error may be syntax errors or logic errors. Syntax

errors can be typographical error or incorrect format of the statement used. Logic errors are caused by incorrect use of control structures.

Documenting the program consists of the written descriptions of what the program is all about and how to use it. The documentation should prepare for the users, operators and the programmers. The steps are as follows:

1. Write user documentation. This is the manual that is prepared to help the user use the programs.
2. Write operator documentation. This manual gives the computer operator information on what to do when the program flashes an error message.
3. Write program documentation. The program documentation consists of the testing of the whole programs. This documentation helps train the new programmers to maintain existing system. Maintenance here means keeping the programs in working conditions, error free and up to date.

PROGRAM LOGIC FORMULATION

Program logic formulation is a step-by-step development of a solution to a given problem. This is the most difficult part of solving a problem with a computer. After you come up with a method of solution, it is a routine to translate your method into the required language, for example C language and any other programming languages. But we must concentrate first how to develop the step-by-step. (This workbook will orient you to C based algorithm).

There are two most used tools that will be of great help for you to formulate a better solution. These are the flowchart and algorithm.

TOOLS USED IN THE DEVELOPMENT

FLOWCHART

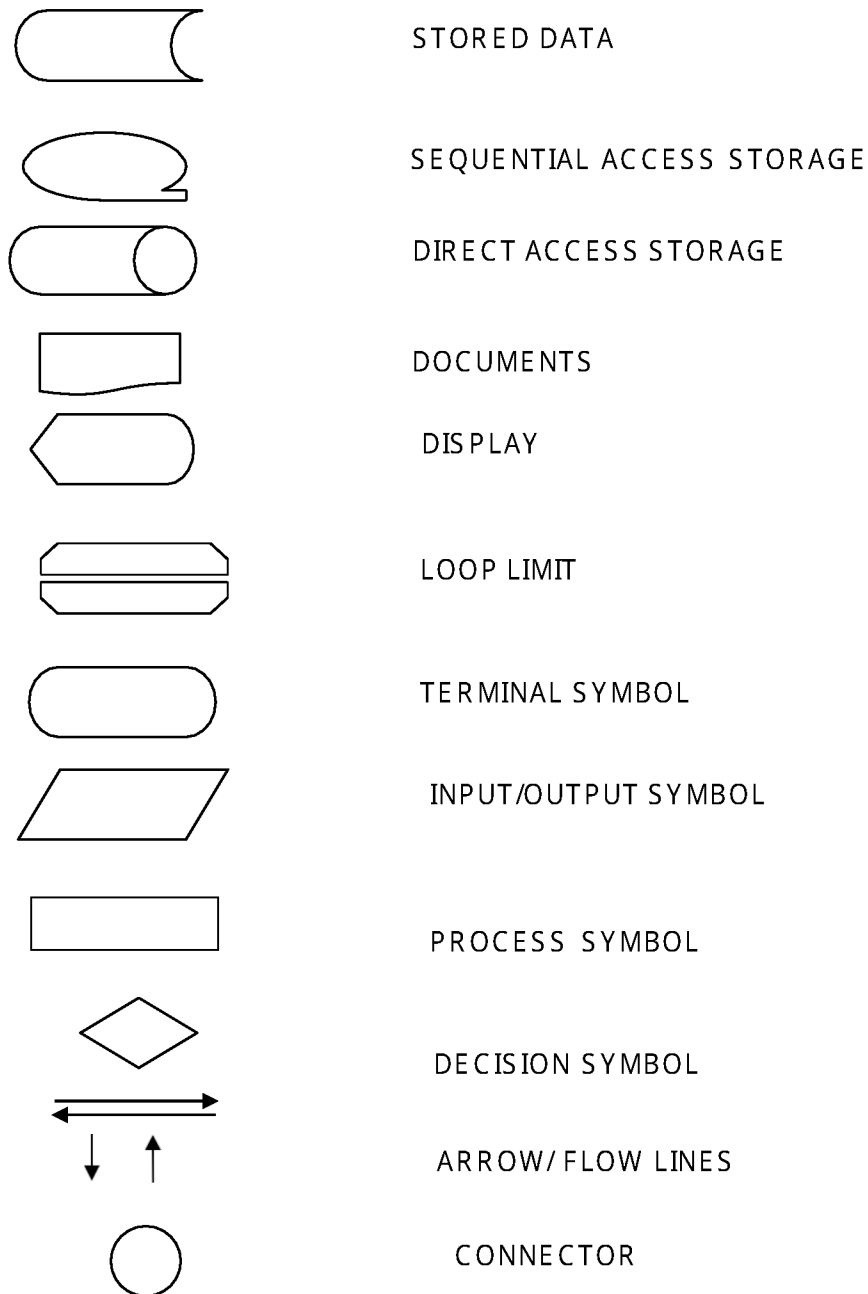
The first tool is a flowchart. It consists of boxes with different shapes that contain simple statements or sentence commonly called instructions and connect together by lines. It is easy to understand because its solution is represented in a picture.

Symbols used in Flowcharts

Data	Indicates Data
Stored Data	Indicates data stored in formats suitable for processing
Sequential Access Storage	Indicates only sequentially accessible data such as those on magnetic tapes & cassette tapes

Direct Access Storage	Indicates directly accessible data, such as those on magnetic disks & floppy disks
Documents	Indicates data on media human beings can read
Display	Indicates data on any medium that displays information used by human beings, such as monitor
Loop Limit	Composed of two parts; condition for termination and loop name are written in the symbol indicating the start & the end of the loop respectively
Terminal Symbol	This oval, which contains either the word start or the word stop, indicates the starting and stopping point in the flowchart. A flowchart can only have one starting point but it may have many possible stopping points.
INPUT/ OUTPUT Symbol	This parallelogram is used for INPUT and OUTPUT instruction. INPUT means to enter data into a computer or to use the computer to read the available data from magnetic disk. OUTPUT means to display the result of compilation or condition of the given solution.
Processing Symbol	This rectangle is used for any data processing operations for which requirements of the problem are usually done here. Examples are computation of average, and assignment of the value to the variable.
Decision Symbol	This diamond-shaped symbol contains a condition. If the condition is true, the marked path TRUE is to be followed. If the condition is false, the marked path FALSE is to be followed.
Arrowheads/Flow lines	Arrows are used to direct the flow of the flowchart. It shows the flow of the solution.
Connector	It is used as a continuation symbol of a flowchart. It shows the flow of the solution.
Initialization/Preparation Symbol	It is used to initialize variables not explicitly defined in the input but a requirement in the production of the output.

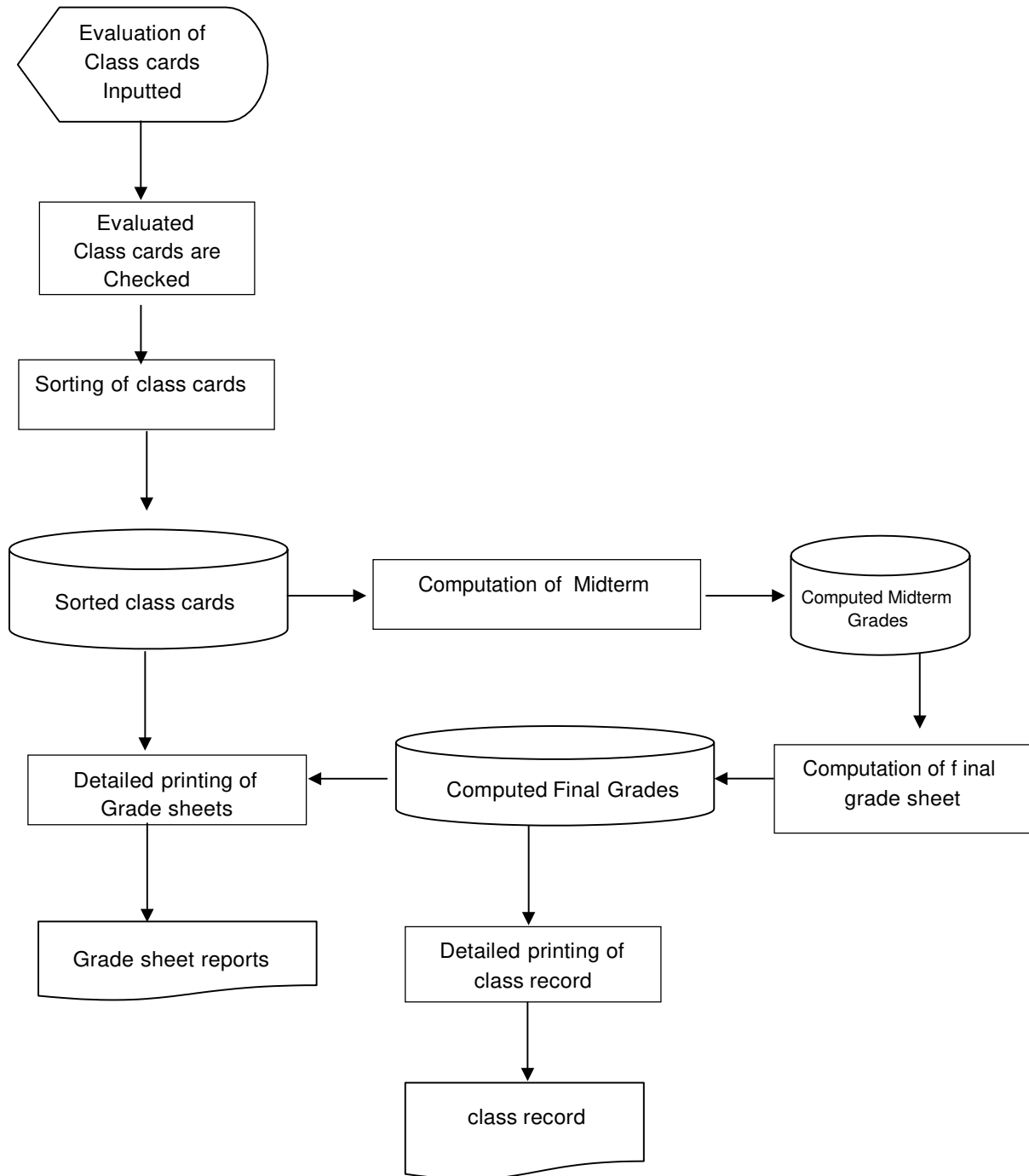
Figure 1 -1. Flowchart Symbols



TYPES OF FLOWCHARTS

- System Flowchart (process chart)
- Program Flowchart

System Flowchart – indicates a flowchart for a targeted system as a whole
A System Flowchart example



Program Flowchart – is used to describe processing procedures based on detailed analysis.

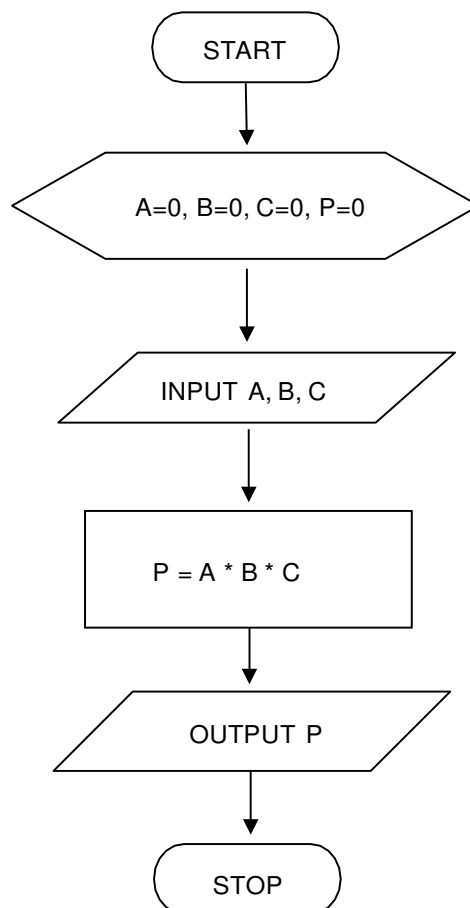
This module will discuss in detail the program flowchart. System flowchart will be discussed thoroughly on System Analysis and Design.

Flowchart Development

The steps that comprise a flowchart must be organized in an orderly, understandable, and clear manner so that the program that implements this flowchart is similarly in an orderly, understandable, and clear manner. An orderly, understandable, and clear solution is called Structured Flowchart. Structured flowcharts are designed using the three most basic methods of control. The basic methods will be our guide to produce a correct structured flowchart necessary to prepare a correct program.

1. **Sequential** – steps are performed in a strictly sequential manner; each step being executed exactly once. This is the simplest method of control. It is easy to develop and understand.

Figure 1-2 shows flowchart on how to determine the product of the three numbers



2. **Selection Structure** – One of several alternative actions is selected and executed. It involves the use of decision based on the given condition. It use decision block or the diamond-shaped block. Alternative action will be represented by a processing block.

Figure 1-3 Flowchart that determines if the average of the student is “PASSING” or “FAILING”

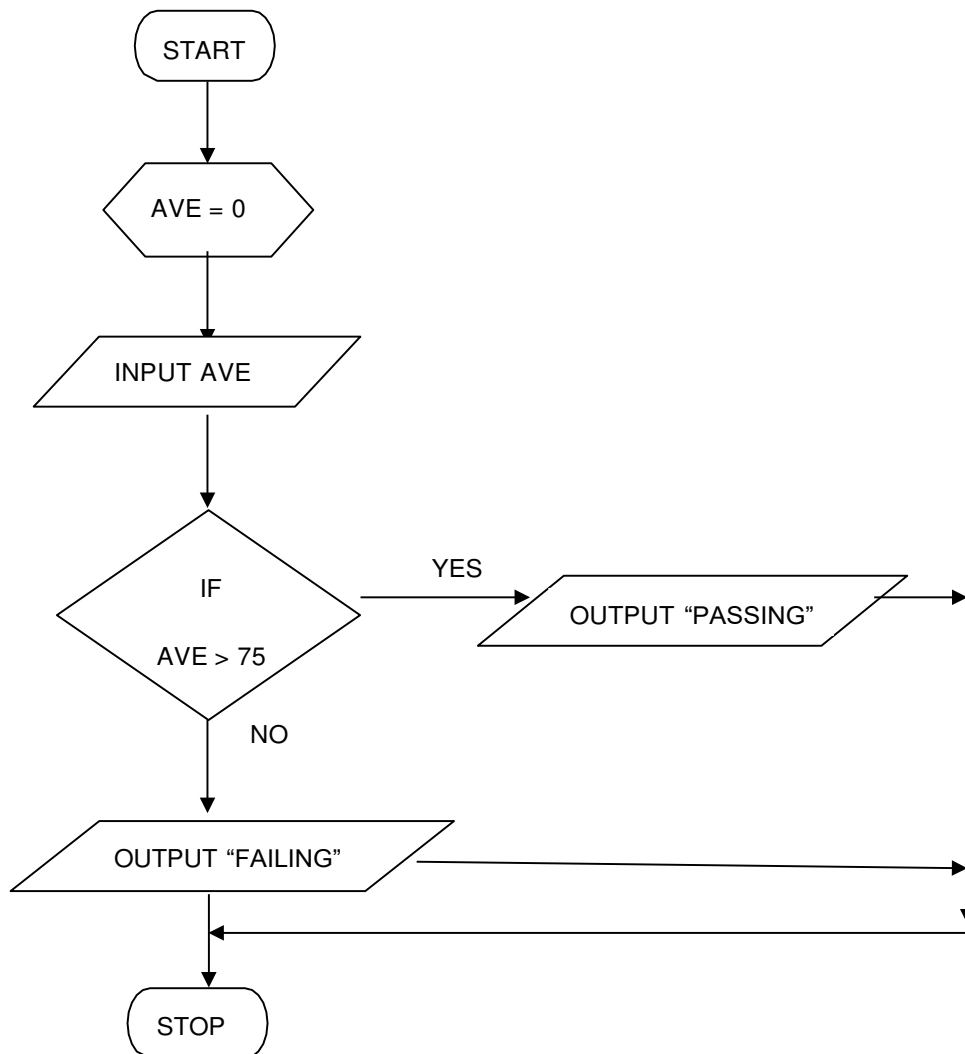
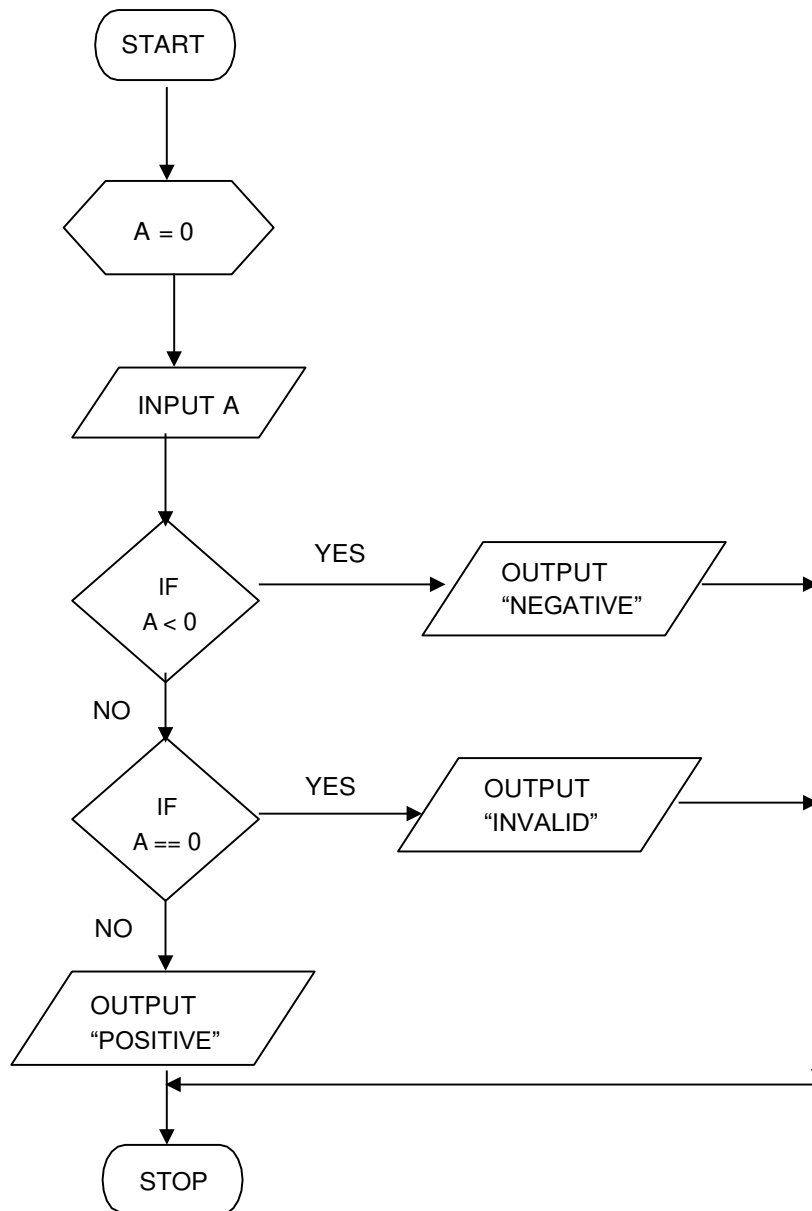


Figure 1-4 shows the flowchart that will determine the positive and negative number.



3. **Repetition Structure** – One or more steps is performed repeatedly.

Figure 1-5 Shows the flowchart for the printing of all odd numbers from 1-10.

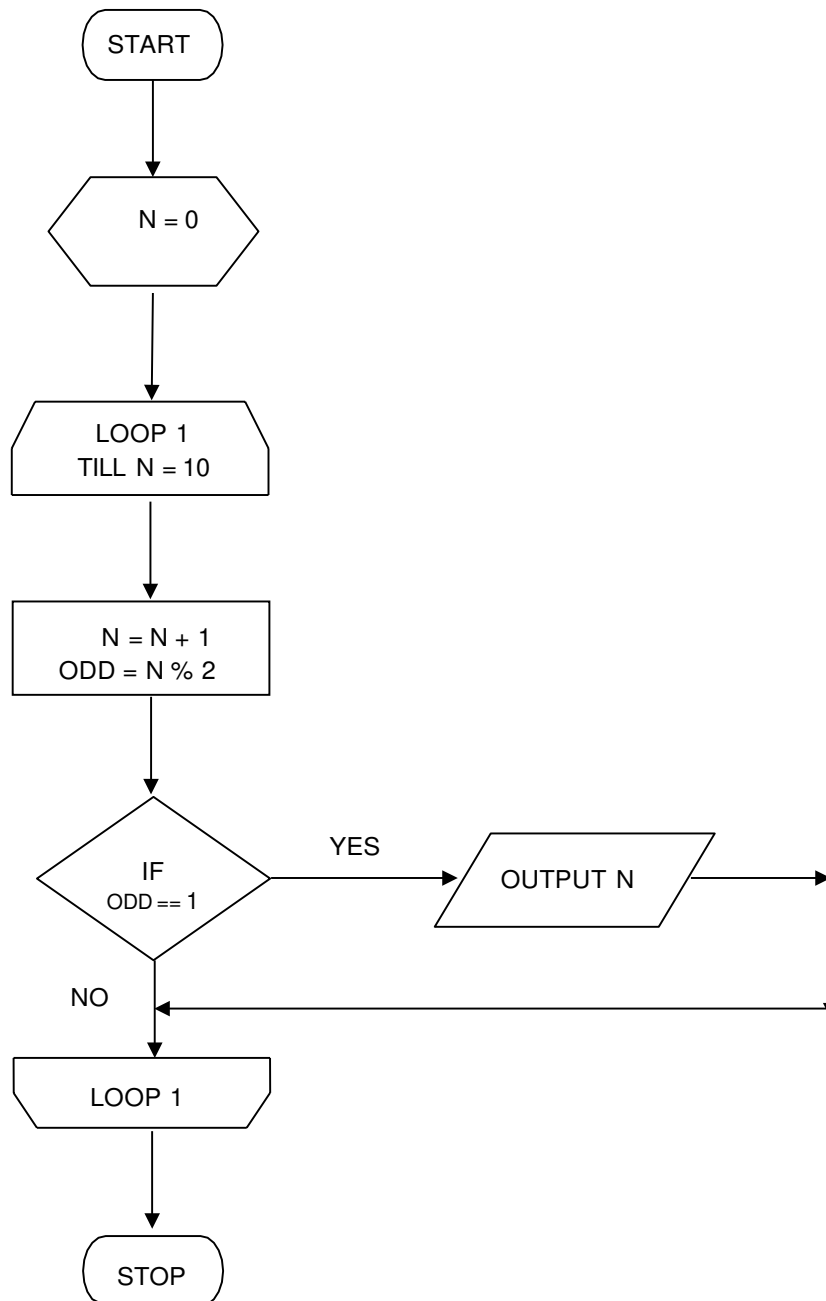
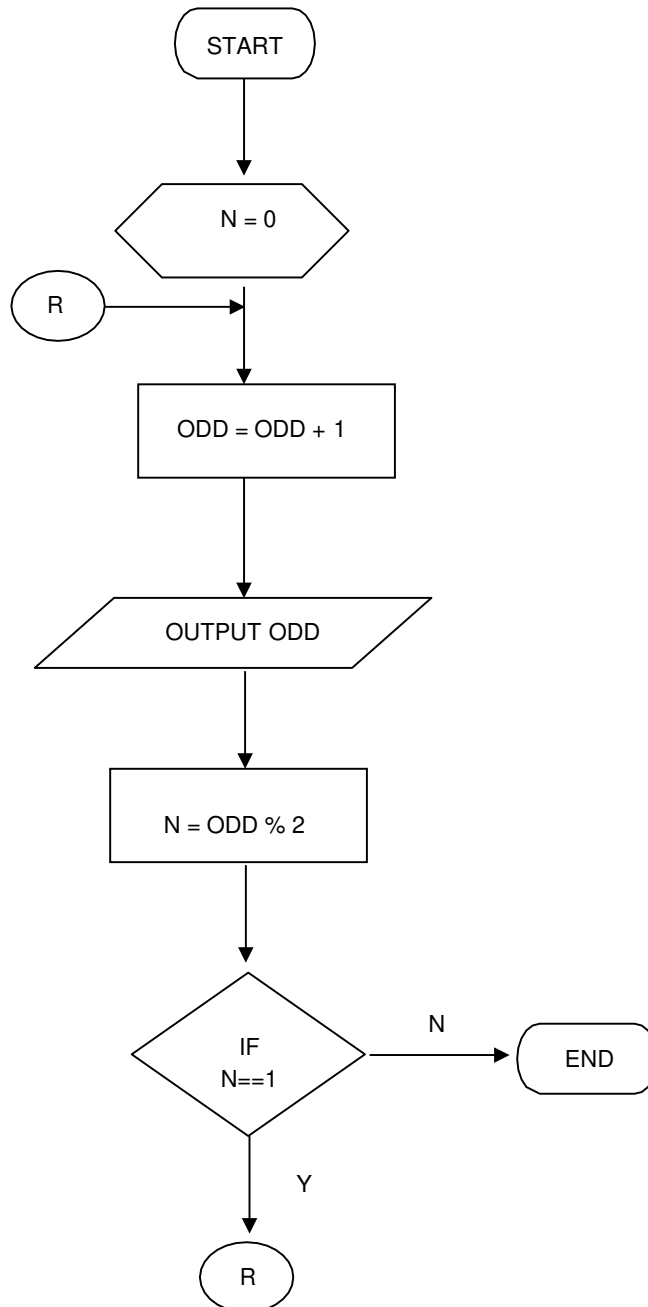


Figure 1-6 Shows the other approach of flowchart for the printing of all odd numbers from 1-10.



These three structures appear to be very simple, but in fact they are sufficiently powerful that any flowchart can be constructed using them.

ALGORITHM

Algorithm is a set or series of instruction for carrying a task. It is also a procedure to produce the required output from the given input. It is similar with a paragraph that expresses a single thought. It uses English language. It is also known as pseudo code.

Algorithm Development

The steps that comprise an algorithm are similar with flowchart. It must be organized in orderly, understandable, and clear manner so that the program that implements this algorithm is similarly orderly, understandable, and clear manner. The three basic methods of control used in a structured flowchart are also applicable in structured algorithm.

1. Sequential
2. Selection
3. Repetition

Pseudo code is a mixture of language and symbols, terms and other feature commonly used one or more high-level languages. High-level languages are programs used by the computer. Typically, the common features of various pseudo codes that appear in textbook are the following:

1. The usual computer symbols are used for arithmetic operations: “+” for addition, “-” for subtraction, “*” for multiplication, and “/” for division.
2. Symbolic names (identifiers) are used to represent quantities being processed by the algorithm.
3. Certain keywords that are common in high-level language may be used, for example, read or enter to indicate input operation; display, print and write for output operations.
4. Indentation is used to set off certain key blocks of instructions.

The structure of an algorithm can be displayed in a structure diagram that shows the various that must be performed and their relation to one another. These diagrams are especially useful in describing algorithms for more complex problem. This section will show similar examples used in flowchart development that will illustrate the three basic control structures – sequential, selection, and repetition and how to present algorithms in pseudo code.

1. Sequential structure

Figure 1-7 Algorithm that shows the printing of the sum

```
ALGORITHM sum
  A -> 0, B -> 0, SUM -> 0
  INPUT A, B
  SUM -> A + B
  OUTPUT SUM

END sum
```

Figure 1-8 Algorithm that shows how to determine the product of three numbers.

```
ALGORITHM product
  A -> 0, B -> 0, C -> 0, PRODUCT -> 0
  INPUT A, B, C
  PRODUCT -> A * B * C
  OUTPUT PRODUCT
END product
```

2. Selection Structure

Figure 1-9 Algorithm that shows how to determine the positive and negative number.

```
ALGORITHM pos_neg
  N -> 0
  INPUT N
  IF N < 0 THEN
    OUTPUT "NEGATIVE"
  END IF
  IF N == 0 THEN
    OUTPUT "INVALID"
  ELSE
    OUTPUT "POSITIVE"
  END IF
END pos_neg
```

Review:
Exercise No. 1

Name: _____
Year&Section: _____

Date: _____
Score: _____

1. Differentiate hardware and software.
2. Differentiate program and programming.
3. What are the components of hardware?
4. What are the types of software?
5. What is data?
6. What is information?
7. What are the five-steps process of programming?
8. What are the following steps necessary in program design?
9. Differentiate syntax error and logic error?
10. To have a clear understanding of the problem, one of the requirements is "Specify the program objectives and identify the users of the program". Explain it briefly.

ANSWERS

1.

2.

3.

4.

5.

6.

7.

8.

9.

10.

Activities/Assessments:

PROGRAMMING 1. Write an algorithm(flowchart and pseudocode for the following problem.

PROGRAMMING EXERCISE 1-1

The volume of a rectangular box is given by the formula, $V = \text{length} * \text{width} * \text{height}$. Design a program that will calculate and display the volume of a box whose dimensions are specified by the user.

PROGRAMMING EXERCISE 1-2

Prepare a program that converts hours to minutes only. A typical conversion should write the message

3 hours is equal to 180 minutes

PROGRAMMING EXERCISE 1-3

Consider three numbers a, b, and c. Write a program that will compute and display their sum, difference, product, quotients (a/ b/ c) and the sum of their squares.

PROGRAMMING EXERCISE 1-4

A supermarket sells bags of mixed candies. Four types of candies are offered. Candy a costs Php. 35.00/kg; Candy B cost Php. 45.00/ kg; candy C cost Php. 56.00/kg and Candy D cost Php. 57.50 kg. Write a program that will calculate and print the amount of bag of candies in terms of the weight of each type of candy as provided by the program user.

PROGRAMMING EXERCISE 1-5

Hypothetically speaking, on Mars a 100-pound person on Earth would weigh 38 pounds. On Jupiter, the same person weight is 264 pounds. Write a program that will input the weight of a person in pound on Earth and displays the corresponding weight when the man is on Mars and on Jupiter.

LESSON 2 Introduction to C

OVERVIEW

This lesson gives a well-rounded overview of the language. C language is a very good language to introduce yourself to the programming world, as it is a simple procedural language which can do wonders. The C language is particularly well suited as an introduction to coding: It is a tried-and-true language, and it allows you to understand computing processes at a deep level.

Objectives

After completion of this Instructional Material you should be able to:

- Know the basic components of Turbo C;
- Explain the basic elements of Turbo C;
- To be able to list, describe, and use the Turbo C basic data types; and
- Enumerate the different C operators.

COURSE MATERIAL

Watch:

[C Programming Tutorial for Beginners](https://www.youtube.com/watch?v=KJgsSFOSQv0)
<https://www.youtube.com/watch?v=KJgsSFOSQv0>
[Fundamentals of C Programming](https://www.youtube.com/watch?v=HV9ba0bNvGQ)
<https://www.youtube.com/watch?v=HV9ba0bNvGQ>

Read:

INTRODUCTION

C is a general-purpose programming language which features economy of expression, modern control flow and data structures and a rich set of operations.

- is has been called as “system programming language” because it is useful for writing compilers and operating systems.

- is a structured programming language. It is considered a high- level language because it allows the programmers to concentrate on the problems at hand.

Brief History of C

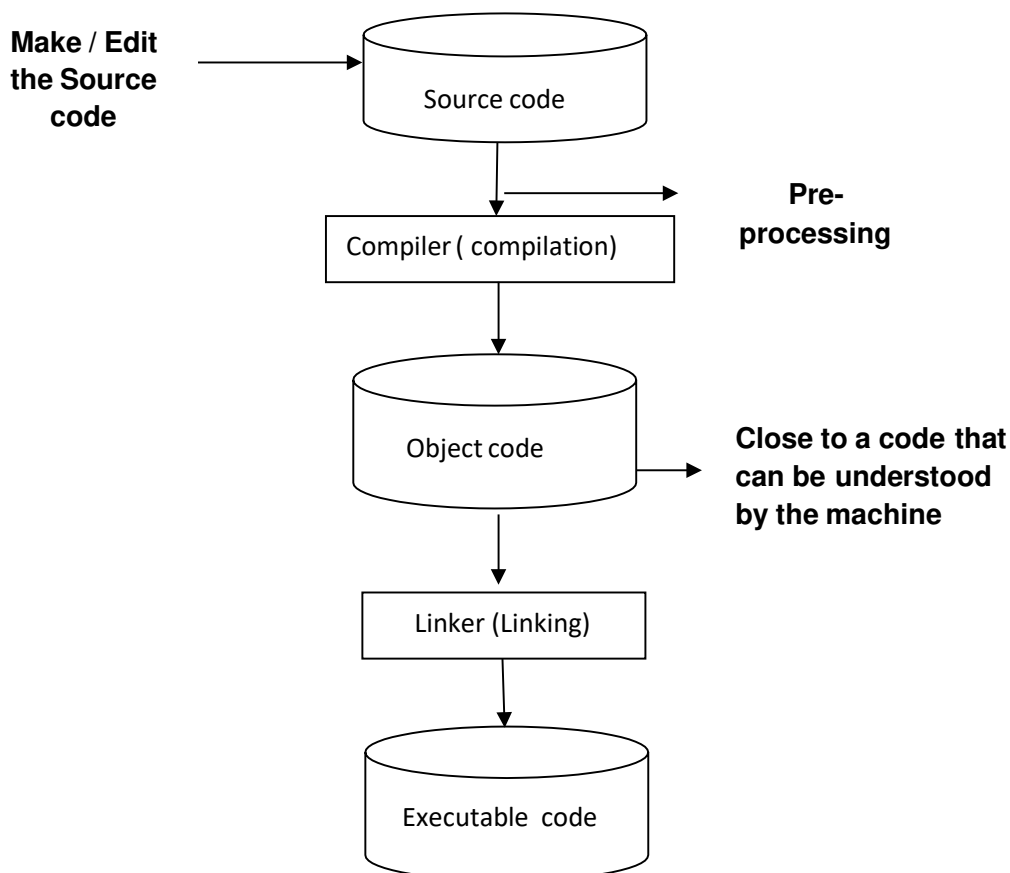
- Basic Combined Programming Language (BCPL) is developed by Martin Richards in 1967. This language is turn strongly influenced the development of the next language –B.
- B is written and developed by Ken Thompson in 1970 for the first UNIX system on the Digital Equipment Corporation (DEC) Programmable Data Processor-7 (PDP-7). Both BCPL and B are “typeless” languages.

- C is an expansion of B. A programming language designed by Dennis Ritchie in 1972 at AT and T Bell Laboratories. C was originally designed for and implemented on the UNIX operating system on the DEC PDP-II. The new C added something that B did not have data types.
- Turbo C is a version of C developed by Borland International Corporation in 1987. This version is designed to run on various microcomputer systems, namely those which use the operating system MS-DOS, the operating system for the IBM Personal Computers and Compatibles.

Definition of Terms:

- **Interpreter** reads the source code of your program one line at a time and performs the specific instructions contained in that line.
- **Compiler** reads the entire program and converts it into object code-the form that can be directly executed by the computer.
- **Compile Time** refers to the events that occur during the compilation process.
- **Object Code** is a translation of the program source code in a form that can be directly executed by the computer. It is also called the binary code and machine code.

1.1 The Programming Process (implemented using C Language)



- **Source Code** is the text of a program that a user can read; commonly thought of as a program.
- **Run Time** refers to the events that occur while the program is executing.
- **Library** is the collection of pre-written.

Syntax errors are detected during compiled time.

Semantic/ Run – Time errors are detected during execution-time.

1.2 Components of Turbo C

Turbo C is more than just a version of the C language. Rather, it includes a complete environment in which to create, test, and run programs. This programming environment consists of several components:

1. **Editor** is used to create program source code.
2. **Extended C language.** This version of C is significantly extended from the “base bone” language of Ritchie’s specifications. The extension includes enhancement which make the Turbo C compatible with the new proposed and ANSI Standard.
3. **Compiler** is used to convert source code into machine code or binary code.
4. **Debugger** is used for testing program and locating programming errors.
5. **Run – Time Environment** is the capability for running programs within the Turbo C system.
6. **User Interface.** The various features of Turbo C are integrated into a single program which allows you to smoothly proceed from source – code entry to compilation to debugging to running without ever leaving the Turbo C environment.

2. The Language Element of C

2.1 Identifiers is names that are used to reference variables, functions, labels, and various other user-defined object; sequence of letters, digits and the special character-, which is called an underscore

2. 1.1 Rules in naming identifiers

1. An identifier in Turbo C can vary from one to several characters; the first character must be a letter or an underscore with subsequent characters being letters, numbers, or the underscore. It may not have a space or a hyphen.
2. Must consist only of alphabetic characters, digits, or underscores.
3. The first 63 characters of an identifier name are significant.
In C, upper-and lower case are treated as different and distinct from one another.

Example: Sname, Sname, and sName are three separate identifiers.

4. An identifier may not be the same as a Turbo C keyword, and it should not have the same name as a function.

2.2 Data Types (Scalar or Simple Data Types)

2.2.1 int variables is whole number consisting of an optional sign (+ or -) followed by a sequence of digit. It cannot contain commas. Variables of this type are often used for controlling loops and conditional statements.

Types of integers

Type	Byte size	Minimum Value	Maximum value
short int	2	-32768	+32767
int	4	-2147483648	+2147483647
long int	4	-2147483648	+2147483647
unsigned int	16	0	65536

2.2.2 float consists of an optional sign (+ or -), followed by one or more digits, a decimal point, and or one or more further digits. It occupies 4 bytes. It can include an optional exponent ranging from 3.4 E to -38 to 3.4 E +38

2.2.3 double (double precision) is a special float which can store more significant digits and have a longer exponent. It occupies 8 bytes in the memory. It ranges from 1.7 E – 308 to 1.7 E +308

2.2.4 char is a single letter, digit, punctuation mark or control symbol recognized by the computer. Written enclosed within single quotation marks, but literals string are enclosed in double quotation marks . A character may be assigned an integer value between -128 and +127. Unsigned char data type may be assigned an integer value from 0 to 255.

Example

```
char c; char B = '*'; char a[30]; a = "apple";
```

2.2.5 void - valueless

Three uses of void:

1. To declare explicitly a function as returning no value.
2. To declare explicitly a function having no parameters.
3. To create generic pointers.

2.3. Variables and Constant Declarations

Variables are identifiers which can be assigned a value within a program. It has 32 valid characters, and these are letters, digits, and underscore.

2.3.1 Variable Declaration

Each variable in program must be declared and defined. In C, a declaration is used to name an object such as variables. Definitions are used to create the object. When we create variables, the declaration gives them a symbolic name and the definition reserves memory for them.

Syntax:

Data-type <variable list>; /* list of variables separated by commas.*/

Example

```
int x, y, z;
char a, b ;
double s;
float ave;
char Sname[30];

int x;
float x;
int y;
int z;

char name[50]="Arnie";
```

2.3.2. Variables initialization

The process of assigning starting values to the variables. It is important to remember that variables are not initialized automatically. When variable is defined they usually contain garbage, so we need to initialize them or store data in them before accessing their values.

2.3.2.1 Several Ways to Initialize a Variables:

1. By assigning an assignment statement (=) may be used to assign values to variables of any types.

syntax: variable-name = expression;

Example

```
x = -1;                      Chl = 'A';
```

2. By using function **scanf**

Example

```
scanf("%d", &x);
scanf("%lf", &y) ;
```

3. By assigning values during declaration:

Example

```
int x =3;
char y = 'x';
double a, b = 100.00;
```

2.3.2 Global and Local Variables

2.3.2.1 Global Variables are initialized only at the start of the program. All global variables are initialized to zero if no other initializer is specified.

2.3.2.2 Local Variables are initialized each time the function in which they are declared is entered.

2.4 Constants refer to fixed values that may not be altered by the program. Turbo C constant can be any of the basic data types or known as Literal Constants.

2.4.1 Character Constants are enclosed between single quotes.

Example

'a', '%%', 'P', 'U', 'P'

2.4.2 Integer constant are specified as number without fractional components.

Example

10, -100, 25, 3179

2.4.3 Floating-point constant required the use of decimal point followed by the number's fractional components.

Example

11.123, 1.25, 5.00

2.4.4 String constant or Literal string constant consists of a phrase contained/enclosed within double quotes. String constants are used in printf statements to introduce text messages into a program.

Example

"PUP", "BSIT", "BSCS"

2.4.5 Declared Constants which are assigned a name Declared constant are defined using a # defined declaration at the beginning of the program. This declaration is introduced by the word # defined follow by a constant name and its value.

Example

# define	total	100
# define	orange	17.50
# define	letter	'M'
# define	x	5
# define	word	"hello"

2.5 Access Modifiers

C has two types of modifiers that are used to control the way in which variables may be accessed or modified.

2.5.1 const

Variables declared under this modifier can not be changed during program execution. However, you may give an initial value before the start of the program.

Syntax:

const < data-type> variable = value;

Example

const float version= 3.20;

This creates a float variable called version that may not be modified by your program. A constant variable will receive its value either from an explicit initialization or by some means dependent on the hardware.

2.5.2 Using # define

The # define declaration is used at the beginning of the program. Like const, # define is also used in declaring constants.

Syntax:

```
define      variable      value
```

Example

```
# define      p- amount      75.35
```

Note: It is not necessary to tell a computer the type of a declared constant. C determines the data type from the assigned value. Remember that although, `const` and `# define` defines a constant, the two modifiers have different syntax.

2.6 Type Modifiers

A **modifier** is used to alter the meaning of the base type to more precisely fit the needs of various situation. With the exception of type `void`, the basic data type may have various modifiers preceding them.

Example

```
signed      unsigned
short      long
```

Possible Combinations of C's Basic Types and Modifiers

Type	Bit Width	Minimum Range	Maximum Range
char	8	-128	127
unsigned char	8	0	255
signed char	8	-128	127
int	16	-32768	32767
unsigned int	16	0	65535
signed int	16	-32768	32767
short int	16	-32768	32767
unsigned short int	16	0	65535
signed short int	16	-32768	32767
long int	32	-2147483648	2147483649
signed long int	32	-2147483648	2147483649
float	64	3.4E-38	3.4E+38
double	64	1.7E-308	1.7E+308
long double	80	3.4E-4932	1.1E+4932

2.7 Assignment Statement

Assignment is the process of assigning a value to a variable. In C, we use the equal sign (=) as the assignment operator.

2.7.1 How to Assign Values to Variables

1. The variable must be on the left side of the equal sign. Values must be on the right side of the equal sign.

2. Any expression can be assigned on the variable.

3. Expressions are not allowed on the left side of the assignment operator.

The assignment operator directs C to assign to the variable on the left side the value on the right side.

Example `X = 31; sum = a + b; Name = "Paolo"; K = 'B';`

2.8 OPERATORS

An operator is a symbol that tells the compiler to perform specific mathematical, relational or logical manipulations.

Three General Classes of C Operators

1. Arithmetic
2. Relational
3. Logical

2.8.1 Arithmetic Operators

* (asterisk)	multiplication
/ (slash)	division
+ (plus)	addition
- (minus)	subtraction
- (unary minus)	negation
% (modulus)	modulus division
++ (plus plus)	increment operator
-- (minus minus)	decrement operator
() (parenthesis)	parenthesis

Important Notes:

1. The operators +, -, *, and / all work the same way in C as they do in most computer languages.
2. When / is applied to an integer or character, any remainder will be truncated.

Example

`11/2 = 5, 10/3 = 3, 13/6 = 2, 3/4 = 0, -34/3 = -11`

3. The modulus division operator, % yields the remainder of an integer division. % cannot be used on type float and double.

Example

`11 % 2 = 1
5 % 3 = 2
147 % 20 = 7
6 % 3 = 0
-25 % 2 = -1`

4. When an increment or decrement operator precedes its operand, C performs the increment or decrement operation prior to using the operand's value. If the operator follows its operand, C uses the operands value before incrementing or decrementing it.

Example

`X = 10;
Y = ++X; X is 11; Y is 11`

```
X = 10;  
Y = X++;
```

X is 11; Y is 10

2.8.1.1 Increment and Decrement Operators

Increment is to add exactly one to the value of a variable.

Decrement is to subtract exactly one from the value of a variable.

++	increment operator	X++	post increment
--	decrement operator	++X	pre increment

Example

1. $X = X + 1$ or $X += 1$ is the same with $X++$

2. $Y = 10, X = 3$

$Z = Y + X++$	$Z = Y + X;$	13
	$X = X + 1;$	4

$Z = Y + ++X$	$X = X + 1;$	4
	$Z = Y + X;$	14

$Z = Y + X--$	$Z = Y + X;$	13
	$X = X - 1;$	2

$Z = Y + --X$	$X = X - 1;$	2
	$Z = Y + X;$	12

2.8.1.2 Types of Arithmetic Operators

1. **Unary Operators** require only a single operand like unary plus and unary minus.

Example

+X, -Y,

- unary plus operator (+) – causes no change to the quantity which follows.
- unary minus operator (-) – causes the sign of the following quantity to be changed.

2. **Binary Operators** requires two operands.

Example

+	(addition)	$X + Y$
*	(multiplication)	$X * Y$

3. **Combined Operators** is C's "shorthand" operation.

Example

$X = X + Y;$

using shorthand can be expressed as $X += Y;$

There are combined operators for all of C's Binary Operators.

Example

Long Hand Notation	(Combined) Shorthand
$X = X + Y;$	$X += Y;$
$X = X - Y;$	$X -= Y;$
$X = X * Y;$	$X *= Y;$
$X = X / Y;$	$X /= Y;$
$X = X \% Y;$	$X \% = Y;$

2.8.2 Relational Operators

Relational refers to the relationship values can have with one another. It is used to determine the relationship of one quantity to another.

Relational Operators Recognized by C

Symbols		Examples
>	greater than	a > b
<	less than	a < b
>=	greater than or equal to	ave >= 90
<=	less than or equal to	ave <=90
=	equal to	a == b
!=	not equal to	a != 100

2.8.3 Logical Operators

Logical refers to the ways the relationships can be connected together using the rules of formal logic.

Logical Operators Recognized by C

&&	and
	or
!	not

Note: In C, true is any value other than 0; false is 0

1	-	true
0	-	false

Example:

```
N = 10 > 5 && !(10 < 9) || 3 <= 4
      1 && !( 0 )      ||      1
      1 && 1          ||      1
                        = 1
```

2.8.4 Ternary Operators

Ternary Operator uses question mark and colon symbols

Syntax:

Expression1 ? expression2 : expression3;

Wherein, expression1, expression2 and expression3 are expressions; expression1 is evaluated, if true then expression2 is evaluated and become the value of the expression; otherwise, expression3 is evaluated and its value becomes the value of the expression.

Example

```
x=1, y = 5;
x>y? sum= x + y: diff = x - y;
```

2.8.5 Comma operator

The comma operator is used to string together several expressions.

Example

```
X= (Y=3, Y++, Y*2); Y=10;
      Y = 3
      Y = 4
      Y = 8
X= (Y= Y-5, 25/Y);
X=5
```

Finally X = 8

2.9 Expressions

Expressions are combinations of operators, constants and variables.

2.9.1 Precedence of Operators

1. Arithmetic ! ++ - -
 * / % + -
2. Relational < <= > >=
 = = ! =
3. Logical &&
 ||
4. Other Operators ? :
 = += -= *= /=

Review:
Exercise No.2

Name: _____

Date: _____

Year§ion: _____

Score: _____

1. Show in a C program statement how the following identifiers can be associated with their respective constant values:

Identifier	Constant Value
a.counter	7 _____
b.length	12 _____
c.offset	12.3723 _____
d.sname	Sonnet _____
e.group	A _____

2. Determine whether the following identifiers are VALID or INVALID:

a. JETT	_____	f. integer	_____
b. gRAde_	_____	g. x_tra	_____
c. float	_____	h. 1311Nov	_____
d. qty_sold	_____	i. Rating	_____
e. _ _	_____	j. @_First!	_____

3. What data type should be declared for the following values?

a. 'A'	_____
b. 32769	_____
c. 32.55	_____
d. 21482.93	_____
e. -123	_____
f. 632179	_____

Activities/Assessment:

Evaluation

2.1. Evaluate the following C arithmetic expressions:

- a. $5 * 6 + 8 - 4 \% 3$ _____
- b. $9 * 2 + (8 - 5) / 2 < 1 * (7 + 4) \&\& 6 + 7 * 4 / 5 > 7 * 2 + 5 - 4$ _____
- c. $6*5/(9-3) != (2*3+10)/4+1 \parallel 8+2*5 < 2*2*5 \&\& 6+2*9 \%4 == 2*3-4$ _____
- d. $6 * 5 / 2 == 15 ? 100 : 200$ _____
- e. $(15 > 26) \parallel !(7 >= 8) \&\& (4 <= 2)$ _____
- false||true&&false
true&&false
false

2.2. Evaluate the following Boolean expressions as TRUE or FALSE.

- a. $2 - 4 > + 3$ _____
- b. $17 == 35$ _____
- c. $22/3 == 5 + 2*2 - 4$ _____
- d. $TRUE \&\& FALSE \parallel TRUE$ _____
- e. $(6 > 3) \&\& (7 != 7)$ _____
- f. $(7 == 2) \parallel (7 > 4) \&\& (6 == 5)$ _____
- g. $7 >= 8$ _____
- h. $5 <= 5$ _____
- i. $7 > 10$ _____

j. $(14 < 2) \parallel (6 < = 5)$

LESSON 3 Input / Output Statements Program Structures

OVERVIEW

In this lesson, we will learn about basic input and output in C programming. We will use the `printf()` and `scanf()` functions to display text on the screen to the user and take user input typed on the keyboard. Input and output is terminology referring to the communication between a computer program and its user. **Input** is the user giving something to the program, and **output** is the program giving something to the user.

Module Objectives

After completion of this Instructional Material you should be able to:

- Understand the importance of C compiler pre-processor;
- Know how to use the input/output statement in Turbo C;
- Differentiate the console input/output statements; and
- Follow the program structure of Turbo C.

COURSE MATERIAL

Watch:

[C Programming: Control Structure](https://www.slideshare.net/sokngimlasv/c-programming-control-structure)

<https://www.slideshare.net/sokngimlasv/c-programming-control-structure>

[C language -Input & Output functions - printf, scanf, getchar, putchar, gets & puts](https://www.youtube.com/watch?v=mmMUmithx_g)

https://www.youtube.com/watch?v=mmMUmithx_g

Read:

**Input / Output Statements
Program Structures**

INTERACTIVE INPUT/OUTPUT

1. C Compiler Pre-Processor

The use of `#define` and `#include`

The C compiler has a preprocessor built into it. Lines that begin with a `#` are called preprocessing directives.

1.1 The `#define`-line can occur anywhere in a program. It affects only the lines in the file that come after it.

Normally, all `# define` are placed at the beginning of the file. By convention, all identifiers that are to be changed by the preprocessor are written in capital letters.

Example

```
# define LIMIT 100
```

```
# define PI    3.14159
```

Explanation:

If the above lines occur in a file that is being compiled, the preprocessor first changes all occurrences of the identifier LIMIT to 100 and all occurrences of PI to 3.14159. The identifier LIMIT and PI are called *symbolic constants*. The use of *symbolic constants* in a program makes it more readable. More importantly, if a constant has been defined symbolically by means of #define facility and used throughout a program, it is easy to change later, if necessary.

1.2 The # include – (commonly known as macro include) is a preprocessing directive that causes a copy of the file to be included at this point in the file when compilation occurs. A #include line can occur anywhere in a file, though it is typically at the head of the file. The quotes surrounding the name of the file are necessary. An include file, is also called “header file”, can contain # define lines and other #include lines. By convention, the names of header files end in .h.

The C system provides a number of standard header files. For example, stdio.h, string.h and math.h. These files contain the declarations of functions in the standard library, macros, structure templates and other programming elements that are commonly used. As we have already seen, the preprocessing directive

```
#include<stdio.h>
```

causes a copy of the standard header file stdoi.h to be included in the code when compilation occurs.

2. Standard Input/Output

2.1 Formatted Output

2.1.1 The printf() function

The function **printf()** is used for output. The “f” in printf() stands for “formatted”. Technically, these functions are not part of the C Language, but rather are part of the C system. They exist in a library and available for use whenever a C system resides. Although the object code for functions in the library is supplied by the C system, it is the responsibility of the programmer to declare the function being used. ANSI C has introduced a new and improved kind of function declaration called *function prototype*. This is one of the most important changes introduced into the language by ANSI C. The function prototypes of functions in the standard library are available in the standard header files. In particular, the function prototypes for printf() and scanf() are in stdio.h. Thus, this header file should be included whenever the function printf() and scanf() is used.

Both printf() and scanf() are passed as list of arguments that can be thought of as control string and other arguments where Control string is a string and may contain conversion specifications or formats. A conversion specification begins with a percent character (%) and ends with a conversion character.

Syntax:

```
printf(“%conversion specification”, variable);
```

Example

```
x=317;
printf("%d", x);

char a[50]="PUP";
printf("%s", a);
```

2.1.2 Conversion Specifications

Conversion Character How the corresponding argument is printed

%c	as a character
%d	as a decimal integer
%e	as a floating point number in scientific notation
%f	as a floating point
%lf	as double
%g	in the e-format of f-format whichever is shorter
%s	as a string

Examples

1. `printf("%c",x);`
2. `printf("%d",y);`
3. `printf("%f",z);`
4. `printf("%s",name);`

When an argument is printed, the place where it is printed is called field and the number of characters in its fields is called its field width. The field width can be specified in a form at as an integer occurring between the percent character (%) and the conversion character.

Example

```
printf("%c %3c %5c\n", 'A', 'B', 'C');
Will print:
A__B      C
```

2.2 Formatted Input

2.2.1 The scanf() function

The function **scanf()** is analogous to the function `printf()` but is used for input rather than output. Its first argument is a control string having formats that corresponds to the various ways the characters in the input stream are to be interpreted. The other arguments are addresses.

Syntax:

```
scanf("%conversion specification", &variable);
```

Example

```
scanf("%d",&x);
```

The format `%d` is matched with the expression `&x`, causing `scanf ()` to interpret characters in the input stream as a decimal integer and to store the result at the address of `x`. Because the

symbol & is the address operator, the expression &x is read as “the address of x”. When the keyboard is used to input values into a program, a sequence of characters is typed, and it is this

sequence of characters, called the input stream, that is received by the program. The `scanf()` function can be used to convert a string of decimal digits into an integer value and to store the value at the appropriate place in memory.

The function `scanf()` function can be used to convert a string of decimal digits into an integer value and to store the value at the appropriate place in memory. The function `scanf ()` returns as an `int` the number of successful conversions accomplished. The function `printf()` returns as an `int` the number of characters printed, or a negative value in case of an error.

2.2.2 Conversion Specification

Conversion Character Valid characters in the input stream

%c	to a character
%d	to a decimal integer
%f	to a floating point number (float)
%lf	to a floating point number (double)
%s	to a string

Examples

1. `scanf("%d",&num);`
2. `scanf("%c",&khar);`
3. `scanf("%s", name);`
4. `scanf("%lf",&duble);`

Sample program using `printf()` and `scanf()`

```
#include<stdio.h>
/* Just a sample program */
main()
{
char c1, c2, c3;
int i;
float x;
double y;

printf("\nInput three characters", "an int, a float, and a double:");
scanf("%c %c %c %d %f %lf",&c1,&c2,&c3,&i,&x,&y);
printf("Here is the data that you typed in:\n");
printf("%3c %3c %3c %5d %12f %12lf",c1,c2,c3,i,x,y);
return 0;
}
```

Explanation

#include<stdio.h> --> including another file

This line tells the computer to include information found in the file `stdio.h`

main() → a function name

C programs consists of one or more “functions,” which are the basic modules of a C program. This program consists of one function called main. The parentheses identify main() as function name.

/* Just a sample program */ → a comment

You can use the symbols /* and */ to enclose comments. Comments are remarks to help clarify a program. They are intended for the reader and are ignored by the computer.

{ → beginning - the body of the function

This opening brace marks the start of the statements that make up the function. The function definition is ended with a closing brace, }.

char c1, c2, c3; → a declaration statement

This statement announces that we will be using variables called c1, c2, c3 and these variables will be of char (character) type. Notice that the statement is terminated with a semicolon. All statements in C program should be terminated with a semicolon.

int i; → a declaration statement

This statement announces that we will be using a variable called i and this variable will be of int (integer) type.

float x; → declaration statements

double y; → declaration statements

These statements announce the use of two variables x,y, variable x will be of type float and variable y will be of type double.

printf(“\n%s\n %s”, “Input three characters”, “an int, a float, and a double:”);
→ a print statement – This prints the phrase within the quotes:

Input three characters
an int, a float, and a double:

scanf(“%c %c %c %d %f %lf”,&c1,&c2,&c3,&i,&x,&y);
an input statement – This reads the values of c1, c2, c3, i, and y and store them in their corresponding addresses.

When reading in numbers, scanf() will skip white space (blank, newlines, and tabs), but when reading in a character, white space is not skipped.

printf(“Here is the data that you typed in:\n”);
printf(“%3c %3c %3c %5d %12f %12lf”,c1,c2,c3,i,x,y);

a print statement – This prints the values which the user may enter.

return 0; → function returns a value unless declared as void

} → end of the function

2.3 The ESCAPE SEQUENCE

<code>\n</code>	-	newline
<code>\t</code>	-	move to the next tab stop
<code>\xhh</code>	-	insert char hh where hh is hexadecimal number
<code>\a</code>	-	sound the bell
<code>\\</code>	-	insert a backslash character.
<code>\"</code>	-	insert a double quote
<code>\'</code>	-	insert a single quote

Laboratory Problem:

1. Input five integers and identify all odd numbers from the numbers.

Input: 10 7 11 21 30

Output:

The odd numbers are: 7 11 21

3. CONSOLE INPUT/OUTPUT (conio.h)

3.1. gets() This function takes the name of the string as an argument and reads characters from the keyboard until ENTER key is pressed. ENTER key is not stored byte is replaces be the null terminator.

3.2. getche() reads a character without echo. Does not wait for carriage return or ENTER key.

3.3 getch() reads a character with echo. Does not wait for carriage return or ENTER key.

3.4. getchar() reads a single individual character from the keyboard and waits for the carriage return or ENTER key.

3.5. puts() writes a string to the screen, followed by a newline. It can only output a string of characters. It cannot output numbers or do format conversion. It takes up less space and run faster.

3.6. putchar() writes a string (character) to the screen.

Sample program 1

```
#include<stdio.h>
#include<conio.h>

/* String example */
main( )
{
    char str[20];

    clrscr();
    puts("Enter any string");
    gets(str);
    puts("You've enter ",str);
```

}

Sample program 2

```
#include<stdio.h>
#include<conio.h>

/* Character example */
main()
{
char str;

clrscr();
while (str != '\n');
str=getchar();
putchar(ch);
}
```

Sample program 3

```
#include<stdio.h>
#include<conio.h>

/* Character example */
main()
{
char str;
clrscr();
str=getch();
putchar(ch);
}
```

4. The General Form of the C program

global declaration/s ;

```
function1( )
{ local variable1;

    statement sequence;
}
```

```
function2( )
{ local variables;

    statements sequence;
}
```

Int a;

```
main( )
{
    local variables;
    statements sequence;
}
```

}

Review :
Exercise No. 3

Name:_____

Date:_____

Year§ion:_____

Score: _____

A.

1. What are header files (#include)? What is the use of stdio.h and conio.h?

2. What is the use of the standard output *printf()*? Standard input *scanf()*?

3. Give the use of the following format specifiers:

%d _____

%c _____

%f _____

%lf _____

4. Give the use of the following backslash character constants:

\n _____

\" (double quotes) _____

' (single quotes) _____

\\ (backslash) _____

5. Write the corresponding C statement to achieve the following output:

a. the sentence "My User Friendly Guide to the C Programming" displayed in the screen

b. read the value of the integer variable *first_int* from the user's input.

c. read the value of a float variable *first_float* from the user's input.

d. display the value of the float variable *first_float*.

e. display the value of the integer variable *first_int*.

6. Describe the following console input/output functions (conio.h):

gets() _____

getch() _____

getche() _____

puts() _____

putchar() _____

System("cls") _____

7. How do we begin and end segment in C?

8. Describe the *General C Language Structure*:

B.

I What will be the output of the following statements:

1. printf("Hello!\n");
2. printf("The value of %5d is five.", 5);
3. printf("\n\n Do you know the next number? \n");
4. printf("\n %5d %6.2f", 6 ,6.5);
5. printf("\Then followed by %d.", d);

II. Consider the following declaration:

```
int x,z;  
char y;  
float t;  
double u;
```

Tell whether the following scanf() statements is valid or invalid.

1. scanf("%d",&x); _____
2. scanf("%f%5d",&t,&z); _____
3. scanf("%c,%c",%y,%y); _____
4. scanf ("%lf",u); _____
5. y=getch(); _____
6. x=getch(); _____

Activities/Assessment:

PROGRAMMING 3. Write a program for the following problem.

PROGRAMMING EXERCISE 3-1

The volume of a rectangular box is given by the formula, $V = \text{length} * \text{width} * \text{height}$. Design a program that will calculate and display the volume of a box whose dimensions are specified by the user.

PROGRAMMING EXERCISE 3-2

Prepare a program that converts hours to minutes only. A typical conversion should write the message

3 hours is equal to 180 minutes

PROGRAMMING EXERCISE 3-3

Consider three numbers a, b, and c. Write a program that will compute and display their sum, difference, product, quotients (a/ b/ c) and the sum of their squares.

PROGRAMMING EXERCISE 3-4

A supermarket sells bags of mixed candies. Four types of candies are offered. Candy a costs Php. 35.00/kg; Candy B cost Php. 45.00/ kg; candy C cost Php. 56.00/kg and Candy D cost Php. 57.50 kg. Write a program that will calculate and print the amount of bag of candies in terms of the weight of each type of candy as provided by the program user.

PROGRAMMING EXERCISE 3-5

Hypothetically speaking, on Mars a 100-pound person on Earth would weigh 38 pounds. On Jupiter, the same person weight is 264 pounds. Write a program that will input the weight of a person in pound on Earth and displays the corresponding weight when the man is on Mars and on Jupiter.

Lesson 4: Program Control Structure

Overview:

Control Structures are just a way to specify flow of control in programs. Any algorithm or program can be clearer and more understood if they use self-contained modules called as logic or control structures. It basically analyzes and chooses in which direction a program flows based on certain parameters or conditions. There are three basic types of logic, or flow of control, known as: Sequence logic, or sequential flow, Selection logic, or conditional flow and Iteration logic, or repetitive flow.

Objectives

After completion of this Instructional Material you should be able to:

- Understand the importance of relational, equality and logical operators used in the flow of control
- Explain the different repetition structure.
- Discuss the conditional statements and the unconditional transfer statements.

Course Materials

Watch:

[C language - Class 9 : Control statements - Part 1 \(If, if-else, nested if, else-if ladder\)](https://www.youtube.com/watch?v=Nm3YYavHtAU)
<https://www.youtube.com/watch?v=Nm3YYavHtAU>

[C language - Class 10 : Control statements - Part 2 \(conditional operator or ternary operator\)](https://www.youtube.com/watch?v=1BP9z9D91DI)
<https://www.youtube.com/watch?v=1BP9z9D91DI>

[C language - Class 11 : Control statements - Part 3 \(goto, break, nested if, else if ladder\)](https://www.youtube.com/watch?v=0HOSYexeZo4)
<https://www.youtube.com/watch?v=0HOSYexeZo4>

[C language - Class 12 : Control statements - Part 4 \(switch case with break statement\)](https://www.youtube.com/watch?v=KhJBsqcGGLo)
<https://www.youtube.com/watch?v=KhJBsqcGGLo>

[Control Structures - while loop - do-while loop - for loop - Goto - break - continue statements](https://www.youtube.com/watch?v=21l11_9Osd0)
https://www.youtube.com/watch?v=21l11_9Osd0

Read: Lesson 4 Program Control Structures

FLOW OF CONTROL

Statements in a program are normally executed one after another. This is called *sequential flow of control*. Often it is desirable to alter the sequential flow of control to provide for a choice of action, or repetition of action. By means of *if*, *if-else*, and *switch statement*, a *selection among alternative actions* are made. By means of *while*, *for* and *do statements*, *interactive actions* can be taken.

The relational, equality, and logical operators are heavily used in flow of control construct.

1. Operators

RELATIONAL, EQUALITY, AND LOGICAL OPERATORS

The following table contains the operators that are most often used to affect flow of control:

1.1 Relational Operators

<	less than
>	greater than
<=	less than or equal to
>=	greater than or equal to

1.2 Equality Operators

=	equal
!=	not equal

1.3 Logical Operators

!	(unary) negation
&&	logical and
	logical or

Relational, Equality, and Logical operators have rules of precedence and associativity that determine precisely how expressions involving these operators are evaluated.

Operators						Associativity
+	(unary) -	(unary) ++	--	!		right to left
*	/	%				left to right
+	-					left to right
<	<=	>	>=			left to right
=	!=					left to right
&&						left to right
						left to right
=	+=	-=	*=	/=	etc	right to left

The ! operator is unary. All other relational, equality, and logical operators are binary. They operate on expressions and yield either the int value 0 or the int value of 1. The reason for this is that in C language false is represented by the value zero and true is represented by any nonzero value. The value for false can be any zero value; or can be a or 0.0 or the null character '\0' or the NULL pointer value. The value true can be any nonzero value.

Example

`a < b` is either true or false. In C, the expression will yield the int value 1 if it is true and the int value 0 if it is false.

1.1.1 Relational Operators and Expressions

The relational operators

`<` `>` `<=` `>=`

are all binary. Each take two expressions as operands and yield either the int value 0 or the int value 1.

Relational expression: `expr < expr`
 `expr > expr`
 `expr <= expr`
 `expr >= expr`

Examples

```
A < 3;  
a > b  
-1.3 >= (2.0 * X + 3.3)
```

1.2.1 Equality Operators and Expressions

The equality operators `==` and `!=` are binary operators acting on expressions. They yield either int value 0 or the int value 1. The usual arithmetic conversions are applied to expressions that are the operands of the equality operators.

equality expression: `expr == expr`
 `expr != expr`

Examples

```
C == 'A';  
K != .2;  
x + y == 3 * z - 7;
```

1.3.1 Logical Operators and Expressions

The logical operator `!` is unary, and the logical operators `&&` and `||` are binary. All of these operators, when applied to expressions, yield either the int value 0 or the int value 1.

Logical negation can be applied to an expression of arithmetic or pointer type. If an expression has value zero, then its negation will yield the int value 0.

logical_negation_expression = !expr;

Examples

```
!a;
```

$$!(x + 6.88);$$

!(a>b) ;

The binary logical operators **&&** and **||** also act on expressions and yield either the int value 0 or the int value 1. The syntax for a logical expression is given by:

Logical_expression → logical_negation_expression;
logical_or_expression;
logical_and_expression;

logical_or_expression → expr || expr;

logical_and_expression → expr && expr;

And			Or		
<i>p</i>	<i>q</i>	$p \cdot q$	<i>p</i>	<i>q</i>	$p \vee q$
<i>T</i>	<i>T</i>	<i>T</i>	<i>T</i>	<i>T</i>	<i>T</i>
<i>T</i>	<i>F</i>	<i>F</i>	<i>T</i>	<i>F</i>	<i>T</i>
<i>F</i>	<i>T</i>	<i>F</i>	<i>F</i>	<i>T</i>	<i>T</i>
<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>

If . . . then			Not	
<i>p</i>	<i>q</i>	$p \supset q$	<i>p</i>	$\sim p$
<i>T</i>	<i>T</i>	<i>T</i>	<i>T</i>	<i>F</i>
<i>T</i>	<i>F</i>	<i>F</i>	<i>F</i>	<i>T</i>
<i>F</i>	<i>T</i>	<i>T</i>		
<i>F</i>	<i>F</i>	<i>T</i>		

Examples

```
a && b;
a || b;
!(a>b) && x
```

2. The COMPOUND STATEMENT

A compound statement is a series of declarations and statements surrounded by braces.

Syntax:

```
compound_statement { { declaration } { statement } }
```

The chief use of the compound statement is to group statements into an executable unit. When declarations come at the beginning of a compound statement, the compound statement is called block. In C, wherever it is syntactically correct to place statement, it is also syntactically correct to place a compound statement. A compound statement is itself a statement. An important use of the compound statement is to achieve the desired flow of control in if, if -else, while, for, and switch statements.

Example

```
#include <stdio.h>
#include <conio.h>
void main( )
{ int a=1, b, c;
  { b=2;
```

```
        c=3;  
    }  
    getch();}
```

3. THE CONDITIONAL STATEMENTS

3.1 The *if statement*

The general form of an *if statement* is:

if (expr)

statement;

Action:

If *expr* is nonzero (true), then *statement* is executed; otherwise *statement* is skipped control passes to the next statement.

Example

```
if (grade >= 90) {  
    printf("Congratulations!!"  
);  
    printf ("Your grade is %d\n", grade);  
}
```

Usually, the expression in an ***if statement*** is relational, logical, equality expression an expression from any domain is permissible.

Where appropriate, compound statement should be used to group a series of statements under the control of a single *if expression*. The code can be written to be more efficient and more understandable by using a single *if statement* with a compound statement for its body.

Example

```
if ( a < b)  
{  
    ave=a;  
    printf("a is smaller than b\n");  
}
```

3.2 The *if-else* statement

The general form is:

```
if (expr)  
    statement1  
;  
else  
    statement2;
```

If *expr* is nonzero, then *statement1* is executed and *statement2* is skipped; if *expr* is zero, then *statement1* is skipped and *statement2* is executed. In both cases control then passes to the next statement.

Example

```
if (a > b)  
{ b++;  
    printf( "Value is smaller %d\n", b);  
}  
else  
    printf("You got a bigger value %d\n", a);
```

An *if-else* statement can be used as the statement part of another *if statement*, this is what you call nested *if*.

Example

```
if(x == 50)
    if(y >= 120)
    { sum = x + y;
      printf("the sum of x and y is %d", sum);
    }

else
{ diff = x - y;
  printf("the difference between x and y is %d", diff);
}
else
    printf("Next time");
```

3.3 THE *switch* STATEMENT

The **switch** is a multiway conditional statement generalizing the if-else statement.

Syntax:

```
switch(expression)
{ case constant expression:
    statement;
    break;
  case constant expression:
    statement;
    break;
  case constant expression:
    statement;
    break;
  [default
    statement;]
}
```

Action:

The **switch** expression maybe any expression which evaluates to an int, conditional, char or double value. The *case list* consists of constants whose type matches that of the *switch* expression. Note that the *case list* cannot include variables or expressions. After the statements for a *case*, a keyword *break* maybe used. The *break* keyword means that at the point, execution should jump to the end of the *switch* statement. The *switch* statement is terminated by curly bracket (`}`). Note the optional *switch* constant *default*. This *switch* constant can be used to specify an action to be taken if the value of the *switch* expression does not match any of the listed values.

Example

Assign a letter grade based on the number value 0-10

```
switch (QUIZ) {
    case 10:
    case 9:    printf("A");
```

break;

```

case      8:    printf("B")
             break;
case      7:    printf("C");
             break;
case      6:    printf("D");
             break;
case      5:
case      4:
case      3:
case      2:
case      1:
case      0:    printf("F"); break;
default:    printf("Input out of Range"); }    →    end of switch

```

Sample Program: This program will prompt the user to input an integer number which will determine the corresponding month in words.

Sample Run:

```

Please enter a number:    11
November

```

```

#include <stdio.h>
#include <conio.h>

```

```

void main()
{ clrscr();
  int number;
  printf("Please enter a number: ");
  scanf("%d",&number);
  switch(number) {
    case 1: printf("January\n");
             break;
    case 2: printf("February\n");
             break;
    case 3: printf("March\n");
             break;
    case 4: printf("April\n");
             break;
    case 5: printf("May\n");
             break;
    case 6: printf("June\n");
             break;
    case 7: printf("July\n");
             break;
    case 8: printf("August\n");
             break;
    case 9: printf("September\n");
             break;
    case 10: printf("October\n");
             break;
    case 11: printf("November\n");

```

```

        break;
    case 12: printf("December\n");
        break;
    default: printf("Invalid entry!"); }
getch();
}

```

4. Unconditional Transfer Statement

THE **break** AND **continue** STATEMENTS

Two special statements: **break();** and **continue;**

4.1 The **break** statement

The **break statement** has two uses:

1. To terminate a CASE in the switch statement
2. To force termination of a loop, bypassing inside the loop, the loop is immediately terminated and program control resumes at the next statement following the loop.

Example

```

while(1) {
    scanf("%lf",&x);
    if(x < 0.0)
        break;
    printf("%f\n",sqrt(x));}

```

4.2 The **continue** statement

The **continue statement** causes the current iteration of the loop to stop and causes the next iteration of the loop to begin immediately. The continue statement may only occur inside for, while, and do loops.

Example

```

do{
    scanf("%d",&num);
    if (x < 0 )
        continue;
    printf("%d",x);
} while (x != 100);

```

5. LOOPS

Repetition of action is one reason we rely on computers. When there are large amount of data, it is very convenient to have a control mechanism that repeatedly execute specific statements. In C, the while, for, and, do statements provide for repetitive actions.

5.1 THE *while* STATEMENT

Syntax:

```
while(expr)
    statement;
next statement;
```

Action:

First the *expr* is evaluated. If it is nonzero(true), then *statement* is executed and control is passed back at the beginning of the while loop. The effect of this is that the body of the *while* loop, namely *statement* is executed repeatedly until *expr* is zero (false). At that point control passes the *next statement*.

It is possible to inadvertently specify an expression that never becomes zero, and unless other means of escaping the while loop are introduced, the program is stuck in an infinite loop. Care should be taken to avoid this difficulty.

Example

```
while(number != 0)
{ scanf ("%d",&number);
  sum += number;
}
printf(" the sum of all numbers is %d",sum);
```

5.2 THE *for* STATEMENT

Syntax:

```
for(initialization; condition; increment)
    statement;
```

Action:

The *for* statement allows many variants, but there are three main parts;

1. **initialization** – is usually an assignment statement that is used to set the loop control variable.
 2. **condition** – is a relational expression that determines when the loop will exit.
 3. **increment** – defines how the loop control variable will change each time the loop is repeated.
- These three major sections must be separated by semicolon. The *for* loop continues to execute as long as the condition is TRUE. Once the condition becomes FALSE, program execution resumes on the statement following the *for* loop.

Example

```
for(x=100;x!=65;x+=5)
{
    z=sqrt(x);
    printf("The square root of %d is %f",x,z);
}
```

An important point above for loop is that conditional test is always performed at the top of the loop. This means that the code inside the loop may not be executed at all if the condition is FALSE to begin with.

5.2.1 *for* LOOP VARIATIONS

5.2.1 One of the most common variations is achieved by using the **COMMA** operator to allow 2 or more variables to control the loop.

Example:

```
for(x=0,y=0; x+y < 10; x++)
{
    scanf("%d", &y);
    sum = x + y;
}
printf("The sum of x and y is %d", sum);
```

5.2.1.1 Nested *for* loop

Syntax:

```
for(initialization1; condition1; increment1)
{
    for(initialization2;condition2;increment2)
    {
        statement;
        statement;
    }
}
```

5.3 THE *do* STATEMENT

Syntax:

```
do {
    Statement;
} while(expr);
next statement;
```

Action:

First *statement* is executed and *expr* is evaluated. If the value of *expr* is nonzero(TRUE), then control passes back to the beginning of the *do* statement and then the process repeats itself. When *expr* is zero (FALSE), then control passes to *next statement*.

Example

```
do {
    scanf("%d",&num);
    sum += num;
} while (num > 100);
printf("Then sum is %d", sum);
```

Sample program:

This program prompts the user to enter series of integer numbers which will be terminated when 0 is entered and will display the sum of all the integer values entered.

```

#include<stdio.h>
#include<conio.h>
void main()
{ int a, sum;
  clrscr();
  printf("Enter series of integers, until 0\n");
  printf("Is entered:\n\n");
  sum=0;

  do {
    printf("enter the number: ");
    scanf("%d", &a);
    sum=sum + a;
  } while (a!=0);
  printf("\nThe sum is %d",sum);
  getch();
}

```

Sample run:

Enter series of integers, until 0 is entered:

Enter the number: 7

Enter the number: 3

Enter the number: 9

Enter the number: 0

The sum is 19

Review
Exercise No. 4

Name: _____

Date: _____

Year and section: _____

Score: _____

1. Give the syntax and explain the following C statements/functions/symbols:
if-else

switch

break

continue

2.

A. Give the syntax and explain for loop:

B. Give the syntax and explain the while loop:

C. Give the syntax and explain the do loop:

3.

Answer the following questions:

1. What is a compound statement?

2. True or false. The else statement in the if.. else statement is required.

3. True or false. All conditional expressions should be enclosed in parenthesis.

4. Correct the syntax error:

```
if x>25.0{  
    y=x  
else  
    y=z;; }
```

5. What value is assigned to num using if statement when var1 is 80?

```
if (var1 > 35)
    num=20.0;
else
    if (var1>50
        num=40.0;
    else
        if(var1>80)
            num = 60.0;
```

6. What will be the output of the following program segment?

(a)

```
x=7; y=8;
if(x<=y)
    if(x==y)
        x++;
    else
        y++;
    printf("%d%d\\",x,y);
```

(b)

```
k=3; j=5;
if (k<4)
    if (j>2)
        if (k-j=3)
            k=k+J
        else
            k=k-j;
    printf("%d\\n",k);
```

7. What output line(s) are displayed by the statements that follow when mark is "T"?
When mark is 'A'? When mark is 'B'? When mark is 'C'?

```
points =0;
switch (mark) {
case 'A':
    points =4;
    exit();
case 'B':
    points =3;
    break;
case 'C':
    points =2;
    continue;
case 'E':
case 'I':
case 'W':
    points =0;
}
```

```

if (points>0)
    printf ("Passed, points earned =%d\n", points);
else
    printf (Failed, no points earned\n");

```

(b)

```

x=7;y=8
if (x<=y)
    {if(x=y)
        x++;}

```

8. Explain the difference between the statements on the left and the statements on the right. For each group of statements, give the final value of x if the initial value of x is 1

```

if (x>0)
    x=x+1;
else if (x>=1)
    x=x+2;

```

```

if (x>0)
    x=x+1;
if (x>=)
    x=x+2;

```

9. Differentiate the use of break and continue.

10. What is the use of else in a switch statement? Is this optional or required?

11. Write a program segment for the following using (a) if statement and (b) switch ..case statement: If the value of the integer variable ME is less than 2 and greater than 0, assign the value of ME to integer variable YOU. If the value of ME is 5, multiply variables THEY and THEM and assign the results to variable YOU. If the value of ME is 2, increment YOU by 1. If the value of ME is 7, decrement YOU by 4. Any other value of ME would display the value of YOU.

12. Write a program segment for the following using case. If the value of the character variable CHAR_VAR is 'A' or 'a', add 1 to integer variable JETT. If the value of CHAR_VAR is 'B' or 'b', subtract Y from Z giving the value to JETT. If the value of CHAR_VAR is 'C' or 'c', subtract Z from Y giving the value to JETT. Any other value would result to an error message "INVALID ENTRY".

13. The following program is partially complete:

```
a=5;
x:
c++;
if(c==1)
{ a=a*a;
  goto x;}
```

```
else
  if (c==2)
  { a = a + a;
    goto x;}
else
```

.....
.....

Complete the program to produce the following output:

The value of a is 25.

The value of a is 10.

The value of a is 0.

14. What is the output of the following program segments:

(a) **for(i=1; i<=5; i++)**
 printf(“%2d”, i);

(b) **ctr = 0;**
 do
 { **ctr = ctr + 1;**
 if ((ctr % 2) != 0)
 continue
 else
 printf(“%2d”, ctr);
 }while (ctr !=10);

(c) **x = 10; y = 10;**
 while (x != y) {
 printf(“%5d %5d\n”,x,y);
 x--; y ++; }

Activities/Assessments

Lesson 4: Program Control Structure

PROGRAMMING. Write a program for the following problem.

PROGRAMMING EXERCISE 4-1

Write a program that will determine whether a person is a child, a teen-ager or an adult using age as input. Assume that a child is from 0 to 12 years old, a teen-ager is from 13

to 19 and adult is from 20 years old and above. Return 'C' from child, 'T' for teen-ager and 'A' for adult.

PROGRAMMING EXERCISE 4-2

Pizza parlors offers different pizza sizes in terms of diameters. For example, Domino's has three options: 10 inches, 12 inches, 14 inches diameter. Extravaganza pizza costing 178 pesos, 259 pesos and 314 pesos respectively. Write a program that will input the diameter of the pizza, and its price. Determine the (a) area of the pizza and (b) the price by the divided area. Find out which of the three options above (for Domino's) is the least expensive in terms of price per square inch of pizza. Note that the area of a circle is equivalent to $PI * (diameter / 2)^2$.

PROGRAMMING EXERCISE 4-3

Write a program that would input 3 integers and would output them in descending order.

PROGRAMMING EXERCISE 4-4

An applicant will be accepted to the Jedi Knight Academy if he is at least 200 cm tall; age is between 21 and 25 inclusive, and a citizen of the Planet Endor. However, if the applicant is recommendee of Jedi Master Obi Wan, he is accepted automatically regardless of his height, age and citizenship. Write a program that would input the applicant's height, age, citizenship code (1- citizen 0-not citizen) and recommendee's code (1- recommended 0- not recommended) and then output whether the applicant is accepted or rejected.

PROGRAMMING EXERCISE 4-5

Assume a range of integer values starting from N1 and ending at N2. Assume also an integer say M. Write a program that will print and display all the numbers from N1 to N2 which are divisible by M. You are also required display the count of such numbers. For example, if N1 = 4, N2 = 12 and M = 4, the output will be: 4, 8, 12. The value 3 will also be displayed indicating that there are 3 numbers which are divisible by 4.

PROGRAMMING EXERCISE 4-6

A Video Rental gives a fine to their customers who return the CDs of tape later than the due date. Input the number of days late and display the fine.

Days	Fine
< = 2	10.00
<= 4	15.00
<= 5	20.00
>= 6	Equal to the amount of rent (CD = 50.00 / VHS = 35.00)

PROGRAMMING EXERCISE 4-7

Write a program that will input non-negative integer and would display the prime factors of the given integer.

Example:

Enter non-negative number: 20

Prime factors are: 1 2 5

CASE STUDY 1

Write a program that accepts a positive integer and gives its prime factorization, that expresses the integer as a product of primes.

CASE STUDY 2

N factorial can be defined as the product of all integers from 1 to N and it is denoted by the symbol N!. 0! (zero factorial) is defined as 1. Write a program that will input N and would display N factorial. (Determine first if N is a nonnegative integer).

CASE STUDY 3

Write a program that will input non-negative integer and call a function DWARF to determine if the integer is DWARF or NOT. An integer is said to be DWARF if the sum of its factors is greater than the half of the number.

Example:

Input number: 6

Factors are: 1 2 3

Sum of its factor: $1 + 2 + 3 = 6$

Half of the number : $6 / 2 = 3$

6 is DWARF

Input number: 9

Factors are 1,3

Sum of its factors : $1 + 3 = 4$

Half of number : $9 / 2 = 4.5$

9 is NOT DWARF